

Crazyflie 2.x Setup Using CFClient, Crazyradio and Xbox Controller

Karan Kumar Anand

July 25, 2026

Contents

1	Objective	2
2	Hardware Used	2
3	Software Used	2
4	Creating the Python Environment	2
4.1	Create Virtual Environment	2
4.2	Activate Virtual Environment	2
5	Installing Required Packages	3
6	Crazyradio Configuration	3
7	Launching CFClient	3
8	Xbox Controller Configuration	4
9	Controller Mapping in CFClient	4
10	Controller Mapping	5
11	Manual Flight Verification	5
12	Problems Encountered	5
13	Solutions Applied	6
14	Outcome	6

1 Objective

The objective of this experiment was to establish manual communication between the Crazyflie 2.x nano quadcopter and a Linux workstation using the Bitcraze CFClient application. The setup involved configuring the Crazyradio USB dongle, connecting an Xbox One S controller, and verifying stable manual flight before transitioning to autonomous control using ROS 2.

2 Hardware Used

- Crazyflie 2.x Nano Quadcopter
- Crazyradio USB Dongle
- Microsoft Xbox One S Wireless Controller (USB Mode)
- Ubuntu Linux Laptop

3 Software Used

- Python 3
- Python Virtual Environment
- CFClient
- cflib
- pygame
- Linux Joystick Utilities

4 Creating the Python Environment

A dedicated Python virtual environment was created to isolate the Crazyflie software packages from the system Python installation.

4.1 Create Virtual Environment

```
python3 -m venv venv
```

4.2 Activate Virtual Environment

```
source venv/bin/activate
```

The terminal prompt changes to

```
(venv)
```

indicating that the virtual environment is active.

5 Installing Required Packages

The required Python packages were installed inside the virtual environment.

```
pip install cfclient
pip install cflib
pip install pygame
```

The installed packages perform the following tasks:

- CFClient provides the graphical interface for controlling the Crazyflie.
- cflib provides the communication library used by CFClient.
- pygame enables joystick support.

6 Crazyradio Configuration

The Crazyradio USB dongle acts as the wireless communication bridge between the computer and the Crazyflie.

Initially, the Crazyradio was detected in bootloader mode instead of normal radio mode. As a result, the Crazyflie could not be discovered by CFClient.

The Crazyradio firmware was updated using the official Bitcraze firmware package. After flashing the firmware, Linux correctly recognized the Crazyradio device.

USB permissions were also configured through udev rules so that the device could be accessed without requiring root privileges.

The device was verified using

```
lsusb
```

which confirmed that the Crazyradio was properly detected.

7 Launching CFClient

After activating the virtual environment, CFClient was started using

```
cfclient
```

The application scanned the available radio interfaces and detected the Crazyflie. The connection was established successfully using the detected radio URI. Successful connection was verified through

- Battery Voltage
- Link Quality
- Sensor Telemetry
- Real-Time Attitude Information

8 Xbox Controller Configuration

The Xbox One S controller was connected through USB.

Before configuring CFClient, Linux was used to verify that the controller was functioning correctly.

```
jstest /dev/input/js0
```

The joystick axes and buttons responded correctly, confirming that the controller drivers were functioning properly.

9 Controller Mapping in CFClient

The following steps were performed inside CFClient.

1. Open CFClient.
2. Navigate to Configure Input Device.
3. Detect the controller.
4. Assign Roll.
5. Assign Pitch.
6. Assign Yaw.
7. Assign Thrust.
8. Save the controller profile.

Initially, the default Xbox controller profiles produced incorrect axis values.

The Roll axis continuously showed an offset of approximately 30° even when the joystick remained untouched.

Several controller profiles were tested.

The PlayStation Mode 3 profile produced the most stable joystick readings.

At rest,

- Roll = 0°
- Pitch = 0°
- Yaw = $0^\circ/s$
- Thrust = 0%

This eliminated unwanted motion caused by joystick calibration errors.

Controller Input	Drone Motion
Left Stick Up	Increase Thrust
Left Stick Left/Right	Yaw
Right Stick Up/Down	Pitch
Right Stick Left/Right	Roll

Table 1: Xbox Controller Mapping

10 Controller Mapping

11 Manual Flight Verification

The Crazyflie was placed on a flat surface before testing.

The following procedure was followed.

1. Verify battery voltage.
2. Ensure joystick values remain centered.
3. Slowly increase thrust.
4. Allow the drone to lift a few centimeters.
5. Verify stable hover.
6. Test Roll, Pitch and Yaw individually.

Successful flight confirmed that

- Crazyradio communication was stable.
- Controller mapping was correct.
- Motor response was normal.
- Communication latency was low.

12 Problems Encountered

Several issues were encountered during the setup process.

1. Crazyradio detected in bootloader mode.
2. Missing USB permissions.
3. Xbox controller not detected correctly.
4. Incorrect joystick mappings.
5. Roll offset of approximately 30°.
6. Axis detection occasionally freezing.
7. Default Xbox controller profiles producing incorrect values.

13 Solutions Applied

The following corrective actions were performed.

- Re-flashed the Crazyradio firmware.
- Configured udev rules.
- Installed pygame.
- Verified joystick operation using Linux joystick utilities.
- Tested multiple controller profiles.
- Selected PlayStation Mode 3 for stable axis values.

14 Outcome

The Crazyflie setup was completed successfully.

The following objectives were achieved.

- Successful Crazyradio communication.
- Successful CFClient connection.
- Stable Xbox controller operation.
- Successful manual flight.
- Verified readiness for ROS 2 integration.

This completed the manual flight setup and established a stable hardware and software platform for subsequent autonomous control using ROS 2.